ЗАКАЗЫ ОБРАБОТКА И АНАЛИТИКА

> Проблема
>
> > Нужно сделать аналитику для заказов из базы данных по реализации. Номера заказов не уникальны. Реализация растянута по времени

1 НОРМАЛИЗУЕМ

1.1 Анализ данных из таблицы sales_saledata

```
SELECT DISTINCT
    client_order, client_order_date, client_order_number
FROM
    sales_salesdata
WHERE
    client_order_number IS NOT NULL
ORDER BY client_order_date
```

Получаем 37 тыс строк

1.2 Смотрим на данные и ищем патерны

client_order:

- NULL
- <Продажи без заказа>
- Заказ клиента ??????? от dd.mm.YYYY HH:MM:SS
- Заказ покупателя ???????? от dd.mm.YYYY HH:MM:SS
- Реализация товаров и услуг КСУТ-?????? от dd.mm.YYYY HH:MM:SS

client_order_number

- Где **client_order:** `is NULL`:
  - Реализация товаров и услуг ????????? от dd.mm.YYYY HH:MM:SS
- Где **client_order:** ilike `<Продажи без заказа>`:
  - Возврат товаров от клиента КСУТ-?????? от dd.mm.YYYY HH:MM:SS
  - Отчет комиссионера (агента) о продажах КСУТ-?????? от от dd.mm.YYYY HH:MM:SS
  - Отчет о розничных возвратах КСУТ-??????? от dd.mm.YYYY HH:MM:SS
  - Отчет о розничных продажах КСУТ-??????? от dd.mm.YYYY HH:MM:SS
- Где **client_order:** ilike `Заказ клиента`:
  - \d+|КСУТ-???????
- Где **client_order:** ilike `Заказ покупателя`:
  - \d+
- Где **client_order:** ilike `Реализация товаров и услуг`:
  - КСУТ-???????

ОТЛИЧНО

Теперь можно приступать к нормализации

Нужно обновить sales_salesdata нормальными `client_order`

```
UPDATE sales_salesdata
SET
  client_order = CASE
    WHEN (client_order IS NULL OR client_order = '<Продажи без заказа>')
         AND client_order_number RLIKE '^Реализация товаров и услуг'
      THEN TRIM(REPLACE(client_order_number, 'Реализация товаров и услуг',
''))

    WHEN (client_order IS NULL OR client_order = '<Продажи без заказа>')
         AND client_order_number RLIKE '^Возврат товаров от клиента'
      THEN TRIM(REPLACE(client_order_number, 'Возврат товаров от клиента',
''))

    WHEN (client_order IS NULL OR client_order = '<Продажи без заказа>')
         AND client_order_number RLIKE '^Отчет комиссионера \\(агента\\) о
продажах'
      THEN TRIM(REPLACE(client_order_number, 'Отчет комиссионера (агента)
о продажах', ''))

    WHEN (client_order IS NULL OR client_order = '<Продажи без заказа>')
         AND client_order_number RLIKE '^Отчет о розничных возвратах'
      THEN TRIM(REPLACE(client_order_number, 'Отчет о розничных
возвратах', ''))

    WHEN (client_order IS NULL OR client_order = '<Продажи без заказа>')
         AND client_order_number RLIKE '^Отчет о розничных продажах'
      THEN TRIM(REPLACE(client_order_number, 'Отчет о розничных продажах',
''))

    ELSE client_order
  END,

  client_order_date = CASE
    WHEN client_order_date IS NULL THEN
      DATE(
        STR_TO_DATE(
          SUBSTRING_INDEX(client_order_number, 'от ', -1),
          '%d.%m.%Y %H:%i:%s'
        )
      )
    ELSE client_order_date
  END
WHERE client_order_number IS NOT NULL
  AND (
    client_order IS NULL OR client_order = '<Продажи без заказа>'
    OR client_order_date IS NULL
  );
```

Добавляем метод `update_sales_with_client_orders()` в скрипт и теперь обновление происходит автоматом

1.3 Делаем модель с заказами в DJANGO в приложении sales

Таблица служебная не для админки

```
class SalesOrders(models.Model):
    client_order = models.CharField(
        max_length=250,
        verbose_name='Заказ клиента'
    )

    client_order_date = models.DateField(
        verbose_name='Дата заказа',
        null=True,
        blank=True
    )

    client_order_number = models.CharField(
        max_length=250,
        verbose_name='Номер заказа клиента',
        null=True,
        blank=True,
        db_index=True
    )

    client_order_type = models.CharField(
        max_length=250,
        verbose_name='Тип заказа',
        null=True,
        blank=True
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=[
                    "client_order",
                    "client_order_date",
                    "client_order_number",

                ],
                name="uniq_sales_order_combo"
            )
        ]
```

Получаем таблицу в базе данных который нам django любезно сделал

- `sales_salesorders` где храним уникальные заказы

Добавляем в `salesdata` fk на `sales_salesorders`

```
orders =
models.ForeignKey(SalesOrders,on_delete=models.CASCADE,null=True,blank=Tru
e,verbose_name='id заказа')
```

1.4 Наполняем таблицы данными

`sales_salesorders`

```
INSERT IGNORE INTO sales_salesorders
  (client_order, client_order_date, client_order_number,
client_order_type)
SELECT DISTINCT
client_order,
client_order_date,
client_order_number,

  CASE
    WHEN client_order_number RLIKE '^(Реализация товаров и услуг|Возврат
товаров от клиента)'
      THEN 'Продажи без заказа'
    WHEN client_order_number RLIKE '^Отчет комиссионера \\(агента\\) о
продажах'
      THEN 'Комиссионные продажи'
    WHEN client_order_number RLIKE '^(Отчет о розничных возвратах|Отчет о
розничных продажах)'
      THEN 'Розничные продажи'
    ELSE 'Заказ клиента'
  END AS client_order_type
FROM sales_salesdata
WHERE client_order IS NOT NULL
  AND client_order_number IS NOT NULL;
```

Круто - справочник заказов создан

Теперь FK в `sales_salesdata`

```
UPDATE sales_salesdata AS t
JOIN sales_salesorders AS s
ON t.client_order <=> s.client_order
AND t.client_order_date <=> s.client_order_date
AND t.client_order_number <=> s.client_order_number
SET t.orders_id = s.id;
```

Связи продажи и заказы созданы. Все норм наверное.

Сразу делаем функцию для автозагрузки справочника на обновление данных `update_salesorders()`

1.4 Как работаем с данными

Получаем все данные по интерисующем заказу в лоб из фактурки

делаем индекс по orders_id чтоб не тормозило

```
SELECT d.*
FROM sales_salesdata d
WHERE d.orders_id = 31745;
```

Получаем все данные по интерисующем заказу но удоваримыми данными из фактурки и справочников

```
SELECT
d.date,
i.fullname,
coalesce(i.article,'нет артикля') as article,
coalesce(b.barcode,'нет баркода') as barcode,
d.dt,
d.cr,
d.quant_dt,
d.quant_cr,
coalesce(mn.name, 'Менеджер не указан') as manager_name,
coalesce(a.name,'Нет агента') as agent_name,
coalesce(d.warehouse,'Склад не указан') as warehouse,
coalesce(sg.name,'') as store_group,
d.spec
FROM sales_salesdata d
JOIN sales_salesorders m ON m.id = d.orders_id
LEFT JOIN corporate_items as i on d.item_id = i.id
LEFT JOIN corporate_barcode as b on d.barcode_id = b.id
LEFT JOIN corporate_managers as mn on d.manager_id = mn.id
LEFT JOIN corporate_agents as a on a.id = d.agent_id
LEFT JOIN corporate_stores as store on store.id = d.store_id
LEFT JOIN corporate_storegroups as sg on sg.id = store.gr_id
where orders_id = 31745;
```

УСЕ 😀

2 ДЕЛАЕМ ВИТРИНЫ И УПРАВЛЕНИЕ В ДЖАНГО МОДЕЛИ ЧТО БЫ КРУТО БЫЛО

Вот запрос на агрегацию и делания витрины / таблицы для ДЖАНГО

```sql
CREATE TABLE IF NOT EXISTS mv_orders
    SELECT
    x.orders_id,
    ord.client_order_type,
    ord.client_order,
    ord.client_order_date,
    min(x.date) as order_min_date,
    max(x.date) as order_max_date,
    DATEDIFF(MAX(x.date), MIN(x.date)) + 1 AS realization_duration,
    DATEDIFF(MAX(x.date),ord.client_order_date) + 1 AS order_duration,
    sum(x.dt) as sales,
    sum(x.cr) as returns,
    sum(x.dt-x.cr) as amount,
    sum(case when parent_id != 86 then x.dt-x.cr else 0 end) as
items_amount,
    sum(case when parent_id = 86 then x.dt-x.cr else 0 end) as
service_amount,
    sum(case when parent_id != 86 then x.quant_dt-x.quant_cr else 0 end)
as items_quant,
    count(distinct(case when parent_id != 86 then x.fullname  else NULL
end)) as unique_items,
    GROUP_CONCAT(DISTINCT manager_name ORDER BY manager_name SEPARATOR ',
') as manager_name
    FROM (
    SELECT
    d.orders_id,
    d.date,
    i.fullname,
    cat.parent_id,
    coalesce(i.article,'нет артикля') as article,
    coalesce(b.barcode,'нет баркода') as barcode,
    d.dt,
    d.cr,
    d.quant_dt,
    d.quant_cr,
    coalesce(mn.name, 'Менеджер не указан') as manager_name,
    coalesce(a.name,'Нет агента') as agent_name,
    coalesce(d.warehouse,'Склад не указан') as warehouse,
    coalesce(sg.name,'') as store_group,
    d.spec
    FROM sales_salesdata d
    LEFT JOIN corporate_items as i on d.item_id = i.id
    LEFT JOIN corporate_cattree as cat on cat.id = i.cat_id
    LEFT JOIN corporate_barcode as b on d.barcode_id = b.id
    LEFT JOIN corporate_managers as mn on d.manager_id = mn.id
    LEFT JOIN corporate_agents as a on a.id = d.agent_id
    LEFT JOIN corporate_stores as store on store.id = d.store_id
    LEFT JOIN corporate_storegroups as sg on sg.id = store.gr_id
    ) x
    join sales_salesorders as ord on ord.id = x.orders_id
    group by x.orders_id
```

```
    order by ord.client_order_date desc;
```

КРУТО МЫ СДЕЛАЛИ MV в mysql. Но обновлять через drop и create пишем функцию `update_mv_orders()` для автообновления и добавляем индексы

ТЕПРЕРЬ ДЖАНГО ВИТРИНУ И АДМИНКУ СО СВИСТЕЛКАМИ И ПЕРДЕЛКАМИ

МОДЕЛЬ

```
class MVSalesOrder(models.Model):
    orders_id = models.BigIntegerField(primary_key=True)
    client_order_type = models.CharField(
        max_length=250, blank=True, null=True, verbose_name="Тип заказов"
    )
    client_order = models.CharField(
        max_length=500, blank=True, null=True, verbose_name="Заказ"
    )
    client_order_number = models.CharField(
        max_length=500, blank=True, null=True, verbose_name="Номер заказа"
    )
    client_order_date = models.DateField("Дата формирования", null=True,
blank=True)
    order_min_date = models.DateField("Первая отгрузка", null=True,
blank=True)
    order_max_date = models.DateField("Последняя отгрузка", null=True,
blank=True)
    order_duration = models.BigIntegerField(
        verbose_name="Срок заказа", blank=True, null=True
    )
    realization_duration = models.BigIntegerField(
        verbose_name="Срок реализации", blank=True, null=True
    )
    sales = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Сумма продаж",
        null=True,
        blank=True,
    )
    returns = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Сумма возвратов",
        null=True,
        blank=True,
    )
    amount = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Сумма заказа",
        null=True,
```

```
        blank=True,
    )
    items_amount = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Стоимость товара",
        null=True,
        blank=True,
    )
    service_amount = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Стоимость услуг",
        null=True,
        blank=True,
    )
    items_quant = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Количество мест в заказе",
        null=True,
        blank=True,
    )
    unique_items = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        verbose_name="Кол-во номенклатур",
        null=True,
        blank=True,
    )
    manager_name = models.CharField(
        max_length=500, blank=True, null=True, verbose_name="Менеджеры"
    )

    class Meta:
        managed = False
        db_table = "mv_orders"
        verbose_name = "Заказ"
        verbose_name_plural = "Заказы"

    def __str__(self):
        return f"{self.client_order}"
```

АДМИНКА + DJANGO DASH + ЧУДЕСА